Filip Pawłowicz, 414324

Regresja liniowa i interakcje

Zbiór danych Real Estate Valuation.

Podstawowym element pracy z danymi jest ich analiza i selekcja cech. Na początku usunąłem kolumnę zawierającą indeksy - do regresji jest to zbędne. Zmieniłem także nazwy kolumn na prostsze i krótsze.

```
df <- read_excel("real-estate.xlsx")
dim(df)

head(df)
summary(df)

df$No <- NULL
names(df) <- c("date", "age", "dist_mrt", "n_stores", "lat", "lon", "price")

head(df)
```

Kolejnym krokiem jest sprawdzenie wartości brakujących lub niepoprawnych. Można to bardzo prosto sprawdzić poleceniem `colSums(is.na(df))`. Ten zbiór danych nie zawiera takich wartości, co wskazali również autorzy zbioru. W ogólnósci brak danych w kolumnie możemy naprawić, podstawiając w te miejsca średnią wartość lub mediane. Jeśli w danej kolumnie takich przypadkach jest więcej, to raczej skłaniamy się do usunięcia tej cechy (lub wierszy).

W modelach liniowych dane nie powinny być silnie skorelowane. Poniżej przedstawiam macierz korelacji oraz stworzoną na jej podstawie tzw. heatmap.

```
cor_matrix <- cor(df)
print(round(cor_matrix, 2))

corrplot(cor_matrix, method = "color", addCoef.col = "black", tl.col = "black")
```

```
          date   age dist_mrt n_stores   lat   lon price
date      1.00  0.02     0.06     0.01  0.04 -0.04  0.09
age       0.02  1.00     0.03     0.05  0.05 -0.05 -0.21
dist_mrt  0.06  0.03     1.00    -0.60 -0.59 -0.81 -0.67
n_stores  0.01  0.05    -0.60     1.00  0.44  0.45  0.57
lat       0.04  0.05    -0.59     0.44  1.00  0.41  0.55
lon      -0.04 -0.05    -0.81     0.45  0.41  1.00  0.52
price     0.09 -0.21    -0.67     0.57  0.55  0.52  1.00
```

Na przedstawionym obrazie widać dużą korelację między longitude a odległością do najbliższej stacji metra. Zbiór danych dotyczy miasta New Taipei, którego

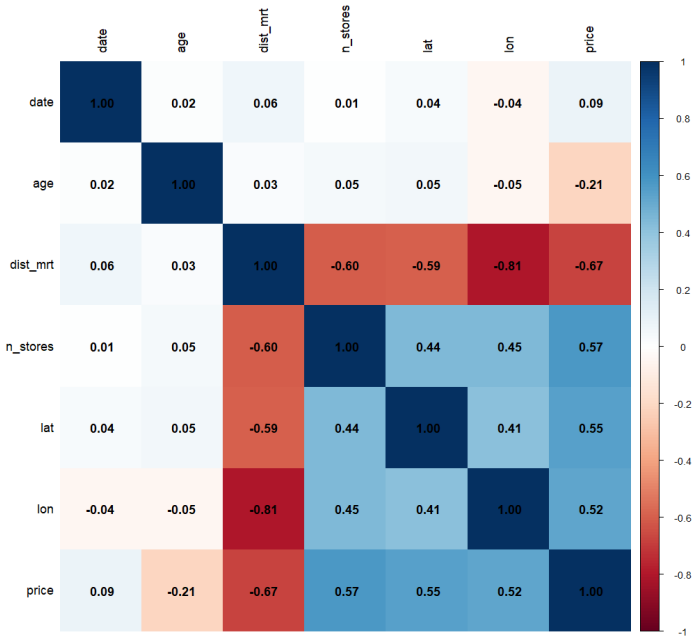


Figure 1: heatmap

centrum zlokalizowane jest bardziej na zachodzie - tam też skupia się większość stacji.

```
plot(df$lon, df$dist_mrt, main = "longitude vs dist_mrt",
     xlab = "longitude", ylab = "dist_mrt", col = "steelblue", pch = 16)
abline(lm(dist_mrt ~ lon, data = df), col = "red", lwd = 2)
```

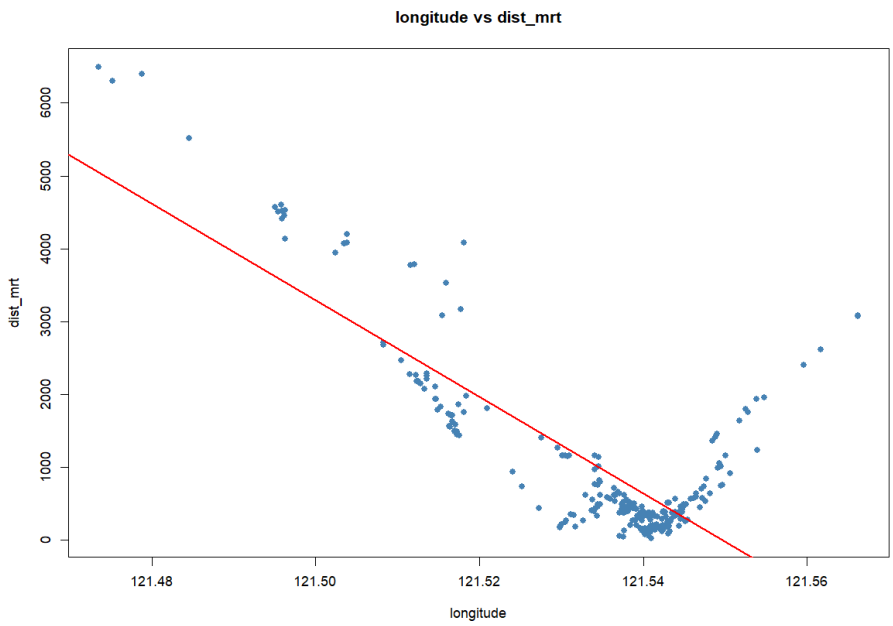
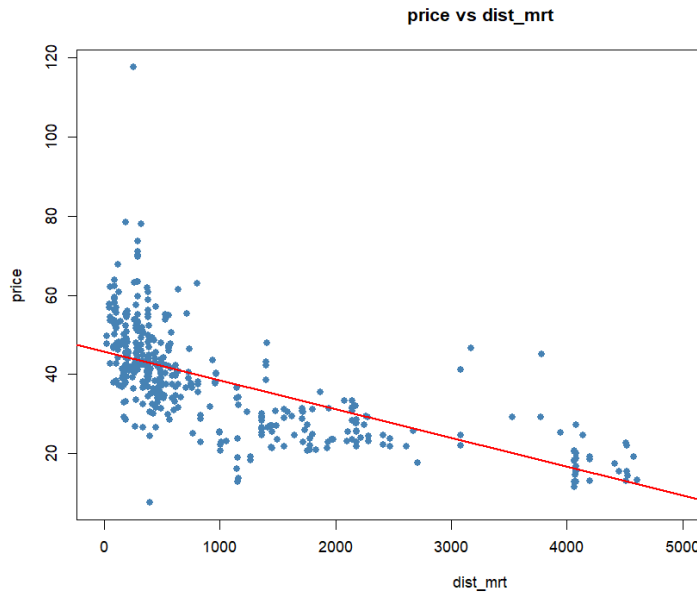


Figure 2: longitude vs dist_mrt

Testem, który służy do sprawdzenia liniowej korelacji danych jest Test Pearsona. W naszym przypadku test zwraca zależność istotnie statystyczną dla podanych cech, ale tylko dla `dist_mrt` i `lon` ta korelacja jest tak duża.

```
cor.test(df$dist_mrt, df$price)
cor.test(df$n_stores, df$price)
cor.test(df$lat, df$lon)
cor.test(df$dist_mrt, df$lon)
```

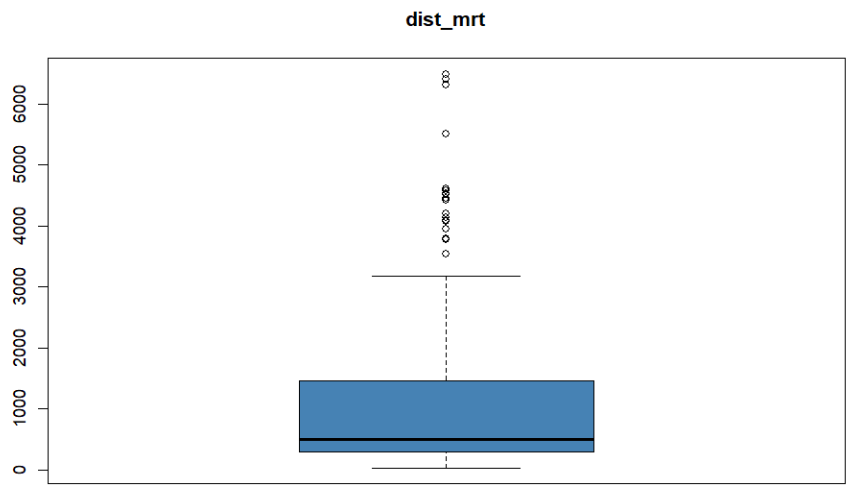
Poniżej przedstawiam też przykładowo wykres ceny od odległości do stacji metra.



Podobne wykresy wykonałem dla pozostałych cech.

Kolejnym niepożądanym zjawiskiem w modelach liniowych są obserwacje odstające. Wynika to z zastosowania MSE jako funkcji kosztu, która silnie karze duże błędy. W praktyce takie obserwacje powinny być usuwane z danych lub odpowiednio ograniczane, np. poprzez winsoryzację, czyli przycinanie wartości skrajnych do ustalonych progów. Zastosowałem drugą opcję. Poniżej przedstawiam kod do stworzenia wykresów pudełkowów oraz jeden przykładowy.

```
boxplot(df$price, main = "price", col = "steelblue")
boxplot(df$dist_mrt, main = "dist_mrt", col = "steelblue")
boxplot(df$age, main = "age", col = "steelblue")
boxplot(df$n_stores, main = "age", col = "steelblue")
```



Na powyższym wykresie pudełkowych widać wiele wartości odstających - są domy, które znajdują się bardzo daleko od stacji metra.

```
winsorize <- function(x) {
  q      <- quantile(x, c(0.25, 0.75))
  iqr    <- q[2] - q[1]
  lower  <- q[1] - 1.5 * iqr
  upper  <- q[2] + 1.5 * iqr
  pmax(pmin(x, upper), lower)
}

df$price    <- winsorize(df$price)
df$dist_mrt <- winsorize(df$dist_mrt)
df$age      <- winsorize(df$age)
```

Stwórzmy teraz model liniowy, sprawdźmy wpływ zmiennych na niego oraz założenia regresji.

```
model_multi <- lm(price ~ ., data = df)
summary(model_multi)

model_interact <- lm(price ~ dist_mrt * n_stores + dist_mrt * lat + age + date, data = df)
summary(model_interact)

AIC(model_multi, model_interact)
```

Na podstawie kryterium AIC widać, że model z interakcjami troszkę lepiej wyjaśnia dane. Będę go stosować w dalszej analizie. Liniowość sprawdzam

wykresem Residuals vs Fitted - reszty powinny być losowo rozrzucone wokół poziomej lini zero, bez żadnego wzorca. U mnie widać lekką krzywą U - powoduje to prawdopodobnie zmienna dist_mrt, która zmienia się dość gwałtownie. Do sprawdzenia autokorelacji cech używam Testu Durbina-Watsona, który sprawdza, czy kolejne reszty są ze sobą skorelowane. Wynik DW jest bliski 2, co oznacza break autokorelacji. Homoskedastyczność testem Breuscha-Pagana - $p > 0.05$, co oznacza wariancję stałą. Testem Shapiro-Wilka sprawdzam, czy reszty mają rozkład normalny - $p < 0.05$, więc nie mają.

```r
plot(model_interact, which = 1)

dwtest(model_interact)

bptest(model_interact)

shapiro.test(residuals(model_interact))
```

```
> dwtest(model_interact)

    Durbin-Watson test

data:  model_interact
DW = 2.1413, p-value = 0.9258
alternative hypothesis: true autocorrelation is greater than 0

> bptest(model_interact)

    studentized Breusch-Pagan test

data:  model_interact
BP = 13.287, df = 7, p-value = 0.06541

> shapiro.test(residuals(model_interact))

    Shapiro-Wilk normality test

data:  residuals(model_interact)
W = 0.92943, p-value = 4.543e-13
```

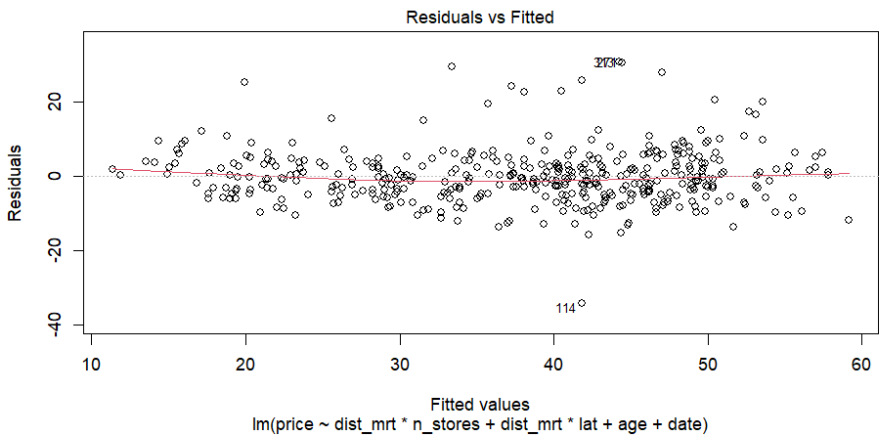


Figure 3: Residuals vs fitted

Regresja Poissonowska

Funkcja wiarygodności dla regresji Poissonowskiej

Dla pojedynczej obserwacji y_i, prawdopodobieństwo że przyjmie ona wartość y_i wynosi:

$$P(Y_i = y_i) = \frac{\lambda_i^{y_i} \cdot e^{-\lambda_i}}{y_i!}$$

gdzie λ_i to oczekiwana liczba zdarzeń dla obserwacji i.

Zakładamy że wszystkie n obserwacji są niezależne, więc łączne prawdopodobieństwo to iloczyn:

$$L(a_0, a_1, \ldots, a_p) = \prod_{i=1}^{n} \frac{\lambda_i^{y_i} \cdot e^{-\lambda_i}}{y_i!}$$

Z założenia:

$$\log(\lambda_i) = a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}$$

czyli:

$$\lambda_i = e^{a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}}$$

$$L(a_0, \ldots, a_p) = \prod_{i=1}^{n} \frac{\left(e^{a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}}\right)^{y_i} \cdot e^{-e^{a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}}}}{y_i!}$$

Maksymalizujemy logarytm funkcji wiarygodności:

$$\ell(a_0, \ldots, a_p) = \sum_{i=1}^{n} \left[y_i \cdot (a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}) - e^{a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}} - \log(y_i!) \right]$$

Niech $\eta_i = a_0 + a_1 X_{i1} + \cdots + a_p X_{ip}$, otrzymujemy postać skróconą:

$$\ell = \sum_{i=1}^{n} \left[y_i \cdot \eta_i - e^{\eta_i} - \log(y_i!) \right]$$

Czynnik $\log(y_i!)$ można pominąć - nie zależy od parametrów.

```
df <- read.table("poisson.data", header = TRUE, sep = ";", dec = ",")

head(df)
summary(df)
dim(df)


likelihood <- function(params, y, x) {
  a0     <- params[1]
  a1     <- params[2]
  eta    <- a0 + a1 * x
  lambda <- exp(eta)
  nll    <- -sum(y * eta - lambda)
  return(nll)
}

names(df)
y <- df$Y
x <- df$X1

result <- optim(par = c(0, 0), fn = likelihood, y = y, x = x)

a0_optim <- result$par[1]
a1_optim <- result$par[2]

cat("a0 (intercept):", a0_optim, "\n")
cat("a1 (slope)    :", a1_optim, "\n")

model_poisson <- glm(y ~ x, data = df, family = poisson)
summary(model_poisson)
```

a) Deviance Residuals to taki odpowiednik reszt z regresji liniowej, ale dla modeli GLM. Mierzy, jak bardzo każda z obserwacji odstaje od wartości

przewidywanej przez model. Liczy się to trochę bardziej skomplikowanym wzorem. W moim p rzypadku prawie wszystkie wartości mieszczą się w przedziale (-2, 2), co oznacza, że model jest dobrze dopasowany.

```
dev_res <- residuals(model_poisson, type = "deviance")
summary(dev_res)
```

```
  Min.  1st Qu.   Median     Mean  3rd Qu.     Max. 
-2.14713 -0.69081  0.08003 -0.02299  0.55972  2.80599 
```

b) Z podsumowania modelu widać, że oba współczynniki mają '***', więc są istotne. Wartości współczynników są takie same, jak te uzyskane przez funkcję optim, gdyż optymalizują tę samą funkcję wiarygodności, róznią się tylko algorytmem.

```
glm(formula = y ~ x, family = poisson, data = df)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)    
(Intercept) 1.032997   0.051017   20.25   <2e-16 ***
x           0.497547   0.006194   80.33   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

c) Null Deviance to przewidywanie każdej obserwacji tylko średnią ze wszystkich y, bez brania pod uwagę predykatora x. Residual deviance to już przewidywanie z braniem predykatora. W moim przypadku residual deviance jest znacznie mniejszy od null deviance, co oznacza, że zmienna x bardzo dużo wyjaśnia.

```
Null deviance: 9945.17  on 99  degrees of freedom
Residual deviance:  101.89  on 98  degrees of freedom
```

Ships Data

W tym przypadku będe modelować częstosć wypadków, czyli liczbę wypadków na jednostkę czasu, a nie samą liczbę wypadków, ze względu na zmienną objaśniająca `period`. Gdy statek jest dłużej eksploatowany, to może mieć więcej wypadków. `period` będzie stanowić tzw. offset w regresji Poissonowskiej.

Zwróćmy też uwage, że zmienne year, period, type to tak naprawdę zmienne kategoryczne, a model potraktuje je jako wartości ciągłe. Wartości tej cechy to po prostu kategorie: 65 oznacza pewien zakres (65-70), a nie konkretną liczbę. W tym celu zamieniamy te zmienne na wartości kategoryczne, stosując One Hot Encoding.

```
library(MASS)
data("ships")
```

```r
head(ships)
summary(ships)
str(ships)

names(ships)

ships$type   <- factor(ships$type)
ships$year   <- factor(ships$year)
ships$period <- factor(ships$period)

ships_df <- subset(ships, service > 0)

model_ships <- glm(incidents ~ type + year + period + offset(log(service)), data = ships_df,

summary(model_ships)
```

```
> summary(ships)
 type  year    period     service          incidents
 A:8   60:10   60:20   Min.   :    0.0   Min.   : 0.0
 B:8   65:10   75:20   1st Qu.:  175.8   1st Qu.: 0.0
 C:8   70:10           Median :  782.0   Median : 2.0
 D:8   75:10           Mean   : 4089.3   Mean   : 8.9
 E:8                   3rd Qu.: 2078.5   3rd Qu.:11.0
                       Max.   :44882.0   Max.   :58.0
```

Z podsumowania modelu widać, że program poprawnie potraktował year i period jako zmienne kategoryczne (osobne zmienne).

```
> summary(model_ships)

Call:
glm(formula = incidents ~ type + year + period + offset(log(service)),
    family = poisson, data = ships_df)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -6.40590    0.21744 -29.460  < 2e-16 ***
typeB       -0.54334    0.17759  -3.060  0.00222 **
typeC       -0.68740    0.32904  -2.089  0.03670 *
typeD       -0.07596    0.29058  -0.261  0.79377
typeE        0.32558    0.23588   1.380  0.16750
year65       0.69714    0.14964   4.659 3.18e-06 ***
year70       0.81843    0.16977   4.821 1.43e-06 ***
year75       0.45343    0.23317   1.945  0.05182 .
period75     0.38447    0.11827   3.251  0.00115 **
---
```

```
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 146.328  on 33  degrees of freedom
Residual deviance:  38.695  on 25  degrees of freedom
AIC: 154.56

Number of Fisher Scoring iterations: 5
```

```r
dev_res <- resid(model_ships, type = "deviance")

hist(dev_res, breaks = 20, col = "steelblue",
     main = "Deviance Residuals - ships",
     xlab = "Value")
abline(v = 0, col = "red", lwd = 2)

qqnorm(dev_res, main = "Q-Q plot residuals")
qqline(dev_res, col = "red", lwd = 2)

shapiro.test(dev_res)
```

```
    Shapiro-Wilk normality test

data:  dev_res
W = 0.94061, p-value = 0.06427
```

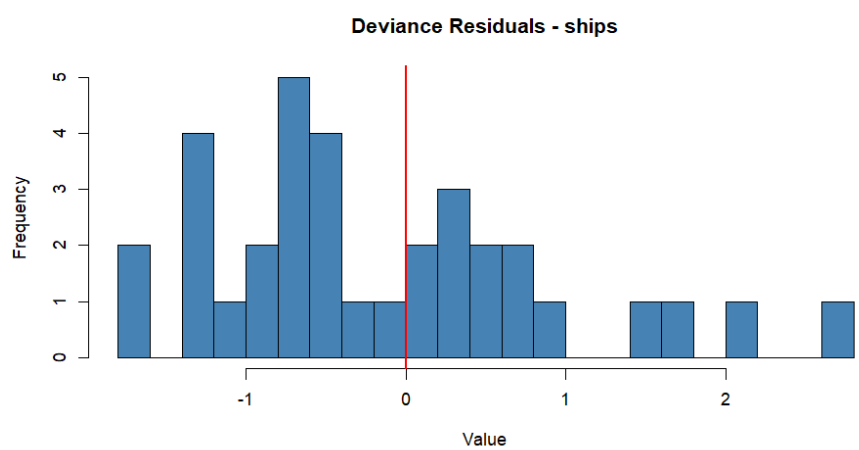


Figure 4: deviance residuals

Tak, jest spełnione założenie reszt dot. ich normalności, co potwierdza Test Shapiro-Wilka.

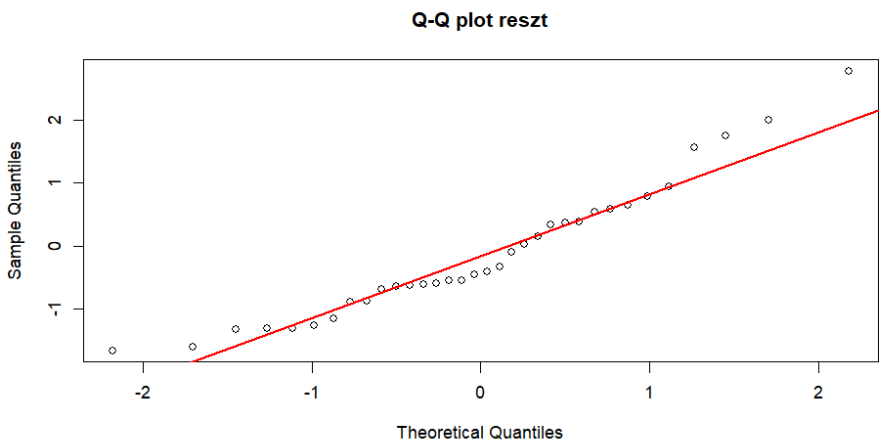


Figure 5: qq plot